Лабораторная работа № 24: Исследование методов создания REST API для работы с пользователями при помощи MongoDB и Koa.js

Цель работы: Сформировать умение анализировать методы создания REST API для работы с пользователями при помощи MongoDB и Koa.js

---

Задание 1 (Базовый уровень)

Создание простого REST API на Koa.js с MongoDB

Описание:
Создать базовый REST API для работы с пользователями: подключение к MongoDB, определение маршрутов, выполнение простых CRUD-операций.

Как это должно работать:

1. Структура проекта:
   - `app.js` — точка входа
   - `db.js` — подключение к MongoDB
   - `routes/users.js` — маршруты пользователей
   - `controllers/users.js` — контроллеры
   - `models/user.js` — модель пользователя
2. Подключение к MongoDB:
   - `MongoClient` инициализируется один раз при старте
   - Клиент передаётся в middleware и доступен через `ctx.db`
   - Коллекция `users` в БД `bbmo_01_23`
3. REST API:
   - `GET /api/users` — список пользователей
   - `GET /api/users/:id` — один пользователь
   - `POST /api/users` — создание
   - `PUT /api/users/:id` — обновление
   - `DELETE /api/users/:id` — удаление
4. Валидация:
   - `name` — обязательное, строка
   - `email` — обязательное, уникальное, формат email
   - `group_name` — обязательное, формат `ББМО-XX-XX`
   - `age` — целое, от 16 до 100
5. Обработка ошибок:
   - Централизованный middleware
   - Ошибки валидации → 400

    - Дубликат email → 409
    - Не найдено → 404
    - Внутренняя ошибка → 500
6. Формат ответа:
    - JSON с полями `data` и `error`
    - Все ответы содержат заголовок `X-Group: ББМО-01-23`

Пример работы:

```
text
```

```
$ npm start
[INFO] MongoDB подключена
[INFO] Koa-сервер на порту 3000
[INFO] Группа: ББМО-01-23

$ curl http://localhost:3000/api/users
{
  "data": [
    {"_id": "...", "name": "Иван", "email": "ivan@example.com",
"group_name": "ББМО-01-23"},
    {"_id": "...", "name": "Мария", "email": "maria@example.com",
"group_name": "ББМО-01-23"}
  ]
}

$ curl -X POST http://localhost:3000/api/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Петр", "email": "petr@example.com", "group_name":
"ББМО-01-23", "age": 20}'
{
  "data": {"_id": "...", "name": "Петр", ...}
}

$ curl -X POST http://localhost:3000/api/users \
  -d '{"name": "П", "email": "invalid"}'
{
  "error": "Validation failed",
  "details": [
    {"field": "name", "message": "минимум 2 символа"},
    {"field": "email", "message": "неверный формат"}
  ]
}

$ curl http://localhost:3000/api/users/999
{
```

```
  "error": "User not found",
  "status": 404
}
```

Что нужно сделать для отчёта:

- Скриншот структуры проекта
- Скриншот запуска сервера
- Скриншот `GET /api/users`
- Скриншот `POST /api/users` (успех)
- Скриншот ошибок валидации
- Скриншот 404
- Скриншот заголовка `X-Group`

Задание 2 (Средний уровень)

Архитектура MVC: модели, сервисы, контроллеры

Описание:
Разделить код на слои: модели (работа с БД), сервисы (бизнес-логика), контроллеры (HTTP). Инкапсулировать SQL/MongoDB-синтаксис в моделях.

Как это должно работать:

1. Слой моделей (`models/user.js`):
    - Класс `User` с методами:
        - `static async findAll(options)` — список
        - `static async findById(id)` — по ID
        - `static async findByEmail(email)` — по email
        - `static async create(data)` — создание
        - `static async update(id, data)` — обновление
        - `static async delete(id)` — удаление
    - Инкапсулирует работу с `MongoClient`
    - Валидация на уровне модели
2. Слой сервисов (`services/userService.js`):
    - Бизнес-логика:
        - Проверка уникальности email
        - Хэширование пароля (если есть)
        - Отправка email-уведомлений (имитация)
        - Логирование операций

    - Вызывает модели
3. Слой контроллеров (`controllers/userController.js`):
    - Валидация входных данных (Joi)
    - Вызов сервисов
    - Установка `ctx.body` и `ctx.status`
    - Обработка ошибок
4. Middleware:
    - `dbMiddleware` — внедрение БД в `ctx.db`
    - `errorMiddleware` — централизованная обработка ошибок
    - `loggerMiddleware` — логирование запросов
    - `validationMiddleware` — валидация через Joi
5. Преимущества архитектуры:
    - Легко тестировать (модели изолированы)
    - Легко менять СУБД (только модели)
    - Легко переиспользовать (сервисы)
6. Схема валидации Joi:
    - `name` — string, min 2, max 100, required
    - `email` — email, required
    - `group_name` — regex `/^ББМО-\d{2}-\d{2}$/`, required
    - `age` — number, min 16, max 100
    - `course` — number, min 1, max 4

Пример работы:

```
text
```

```
$ npm start
[INFO] MongoDB подключена
[INFO] Koa-сервер на порту 3000

$ curl -X POST http://localhost:3000/api/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Иван", "email": "ivan@example.com", "group_name":
"ББМО-01-23", "age": 20}'
[2024-01-15 14:30:25] [SERVICE] Создание пользователя: ivan@example.com
[2024-01-15 14:30:25] [MODEL] insertOne
[2024-01-15 14:30:25] [SERVICE] Email отправлен: ivan@example.com
{
  "data": {"_id": "...", "name": "Иван", "email": "ivan@example.com"}
}

$ curl -X POST http://localhost:3000/api/users \
  -d '{"name": "Иван", "email": "ivan@example.com", ...}'
[2024-01-15 14:30:30] [SERVICE] Проверка уникальности email
```

```
[2024-01-15 14:30:30] [SERVICE] Email уже существует
{
  "error": "Email already exists",
  "status": 409
}
```

Что нужно сделать для отчёта:

- Скриншот структуры проекта (папки models, services, controllers)
- Скриншот работы модели `User`
- Скриншот работы сервиса `userService`
- Скриншот работы контроллера
- Скриншот middleware (db, error, logger, validation)
- Скриншот ошибки валидации Joi
- Скриншот проверки уникальности email

Задание 3 (Продвинутый уровень)

Валидация, пагинация, фильтрация, сортировка

Описание:
Расширить API: добавить пагинацию, фильтрацию, сортировку, поиск, а также строгую валидацию всех входных данных.

Как это должно работать:

1. Пагинация:
    - Query-параметры: `?page=1&limit=10`
    - Возврат метаданных: `total`, `page`, `limit`, `pages`
    - По умолчанию: `page=1`, `limit=10`
    - Максимум: `limit=100`
2. Фильтрация:
    - По группе: `?group_name=ББМО-01-23`
    - По курсу: `?course=2`
    - По возрасту: `?age_min=18&age_max=25`
    - Комбинирование фильтров
3. Сортировка:
    - `?sort=name` — по имени (A-Z)
    - `?sort=-name` — по имени (Z-A)
    - `?sort=age` — по возрасту

        - `?sort=-created_at` — по дате создания
    4. Поиск:
        - `?search=Иван` — поиск по имени и email
        - Использование текстового индекса MongoDB
        - Ранжирование по релевантности
    5. Валидация query-параметров:
        - `page`, `limit` — целые, положительные
        - `sort` — из белого списка полей
        - `group_name` — формат `ББМО-XX-XX`
        - Joi-схема для query
    6. Расширенные эндпоинты:
        - `GET /api/users` — с пагинацией, фильтрами, сортировкой
        - `GET /api/users/search?q=...` — поиск
        - `GET /api/users/stats` — статистика
        - `GET /api/users/export` — экспорт в CSV
    7. Статистика (агрегация MongoDB):
        - Общее количество
        - Средний возраст
        - Количество по группам
        - Количество по курсам

Пример работы:

```
text
```

```
$ curl "http://localhost:3000/api/users?page=2&limit=5&sort=-age"
{
  "data": [...],
  "pagination": {"total": 100, "page": 2, "limit": 5, "pages": 20}
}

$ curl "http://localhost:3000/api/users?group_name=ББМО-01-23&course=2"
{
  "data": [...],
  "pagination": {"total": 25, "page": 1, "limit": 10, "pages": 3}
}

$ curl "http://localhost:3000/api/users/search?q=Иван"
{
  "found": 15,
  "data": [{"name": "Иван Иванов", "score": 1.5}, ...]
}

$ curl "http://localhost:3000/api/users/stats"
```

```
{
  "total": 100,
  "averageAge": 20.5,
  "byGroup": {"ББМО-01-23": 40, "ББМО-02-23": 35, "ББМО-03-23": 25},
  "byCourse": {"1": 30, "2": 35, "3": 20, "4": 15}
}

$ curl "http://localhost:3000/api/users?page=0"
{
  "error": "Validation failed",
  "details": [{"field": "page", "message": "должно быть больше 0"}]
}
```

Что нужно сделать для отчёта:

- Скриншот пагинации
- Скриншот фильтрации
- Скриншот сортировки
- Скриншот поиска
- Скриншот валидации query-параметров
- Скриншот статистики (агрегация)
- Скриншот экспорта CSV

Задание 4 (Сложный уровень)

Аутентификация, авторизация и безопасность

Описание:
Добавить в API аутентификацию через JWT, авторизацию по ролям, хэширование паролей и защиту от основных атак.

Как это должно работать:

1. Регистрация и вход:
    - `POST /api/auth/register` — регистрация
    - `POST /api/auth/login` — вход, возврат JWT
    - `POST /api/auth/refresh` — обновление токена
    - `POST /api/auth/logout` — выход
2. JWT:
    - Access token (15 минут)
    - Refresh token (7 дней)

    - Payload: `{ userId, email, role }`
    - Секрет в `.env`
    - Подпись HS256
3. Хэширование паролей:
    - `bcrypt` с 10 раундами
    - Пароль не хранится в открытом виде
    - Проверка через `bcrypt.compare()`
4. Авторизация по ролям:
    - Роли: `user`, `admin`, `moderator`
    - Middleware `requireRole('admin')`
    - `GET /api/admin/users` — только админ
    - `DELETE /api/users/:id` — только админ или владелец
5. Защита маршрутов:
    - `authMiddleware` — проверка JWT
    - Внедрение `ctx.user` из токена
    - Публичные маршруты: `/api/auth/*`, `GET /api/users`
    - Защищённые: `POST/PUT/DELETE /api/users/*`
6. Безопасность:
    - Helmet (заголовки безопасности)
    - CORS (белый список)
    - Rate limiting (100 запросов/мин на IP)
    - Валидация всех входных данных
    - Защита от NoSQL-инъекций (проверка типов)
    - Логирование всех попыток входа
7. Обработка ошибок:
    - Неверный токен → 401
    - Недостаточно прав → 403
    - Истёкший токен → 401 с указанием обновить
    - Неверный пароль → 401 (без деталей)

Пример работы:

```
text

$ curl -X POST http://localhost:3000/api/auth/register \
  -d '{"name": "Иван", "email": "ivan@example.com", "password":
"secret123"}'
{
  "data": {"userId": "...", "email": "ivan@example.com"},
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi..."
}

$ curl -X POST http://localhost:3000/api/auth/login \
```

```
  -d '{"email": "ivan@example.com", "password": "secret123"}'
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "expiresIn": 900
}

$ curl -X POST http://localhost:3000/api/users \
  -H "Authorization: Bearer eyJhbGciOi..." \
  -d '{"name": "Петр", ...}'
{
  "data": {"_id": "...", "name": "Петр"}
}

$ curl -X POST http://localhost:3000/api/users \
  -d '{"name": "Петр", ...}'
{
  "error": "Unauthorized",
  "status": 401
}

$ curl http://localhost:3000/api/admin/users \
  -H "Authorization: Bearer <user-token>"
{
  "error": "Forbidden: admin role required",
  "status": 403
}

$ curl -X POST http://localhost:3000/api/auth/login \
  -d '{"email": "ivan@example.com", "password": "wrong"}'
{
  "error": "Invalid credentials",
  "status": 401

}
```

Что нужно сделать для отчёта:

- Скриншот регистрации
- Скриншот входа (получение JWT)
- Скриншот защищённого запроса
- Скриншот 401 без токена
- Скриншот 403 при недостатке прав
- Скриншот обновления токена
- Скриншот хэширования пароля (в БД)

- Скриншот rate limiting
- Скриншот логов попыток входа

Задание 5 (Экспертный уровень)

Полноценный REST API с документацией, тестами и CI/CD

Описание:
Создать production-ready REST API с автоматической документацией Swagger, юнит- и интеграционными тестами, CI/CD и мониторингом.

Как это должно работать:

1. Структура проекта:
    - `src/` — исходный код
    - `tests/` — тесты
    - `docs/` — документация
    - `migrations/` — миграции
    - `scripts/` — скрипты
    - `docker/` — Docker-конфигурация
    - `.github/workflows/` — CI/CD
2. Документация (Swagger/OpenAPI):
    - `swagger-jsdoc` + `swagger-ui-express`
    - JSDoc-комментарии в контроллерах
    - Автогенерация спецификации
    - Доступна по `/api-docs`
    - Интерактивное тестирование
3. Тестирование:
    - Юнит-тесты: сервисы изолированно (Jest)
    - Интеграционные тесты: API через `supertest`
    - E2E-тесты: полный цикл с реальной БД (mongodb-memory-server)
    - Покрытие > 80%
    - `npm test` и `npm run test:coverage`
4. CI/CD (GitHub Actions):
    - Запуск линтера (ESLint)
    - Запуск тестов
    - Проверка покрытия
    - Сборка Docker-образа
    - Публикация в registry
    - Деплой на staging

5. Мониторинг:
    - Prometheus-метрики (`/metrics`)
    - Healthcheck (`/health`)
    - Readiness (`/ready`)
    - Логи в JSON (winston/pino)
    - Трейсинг (опционально)
6. Docker:
    - `Dockerfile` для приложения
    - `docker-compose.yml` с MongoDB
    - Multi-stage build
    - Минимальный образ (alpine)
    - Переменные окружения
7. Расширенные возможности:
    - WebSocket для уведомлений
    - Email-уведомления (nodemailer)
    - Загрузка аватаров (multer)
    - Экспорт данных (CSV, JSON)
    - Импорт данных
    - Soft delete (флаг `deleted_at`)
8. Безопасность:
    - Helmet, CORS, Rate limiting
    - JWT + Refresh tokens
    - Хэширование паролей (bcrypt)
    - Валидация (Joi)
    - Защита от NoSQL-инъекций
    - HTTPS в production
    - Секреты через env

Пример работы:

```
text
```

```
$ docker-compose up
[INFO] MongoDB запущена
[INFO] Приложение запущено на порту 3000
[INFO] Swagger: http://localhost:3000/api-docs
[INFO] Metrics: http://localhost:3000/metrics
[INFO] Health: http://localhost:3000/health

$ curl http://localhost:3000/health
{"status": "ok", "uptime": 3600, "db": "connected"}

$ curl http://localhost:3000/api-docs
(HTML-страница Swagger UI)
```

```
$ npm test
PASS  tests/unit/userService.test.js
PASS  tests/integration/users.test.js
PASS  tests/e2e/auth.test.js
Tests: 45 passed, 45 total
Coverage: 87%

$ curl http://localhost:3000/metrics
# HELP http_requests_total
# TYPE http_requests_total counter
http_requests_total{method="GET",path="/api/users",status="200"} 1523
...

# CI/CD
✓ Lint
✓ Tests (45 passed)
✓ Coverage (87%)
✓ Build Docker
✓ Push to registry
✓ Deploy to staging
```

Что нужно сделать для отчёта:

- Скриншот структуры проекта
- Скриншот Swagger UI
- Скриншот `GET /health`
- Скриншот `GET /metrics`
- Скриншот результатов тестов
- Скриншот покрытия
- Скриншот CI/CD (GitHub Actions)
- Скриншот `docker-compose up`
- Скриншот WebSocket-уведомлений

Фиксация изменений

Добавить файлы в Git
Создать коммит
Отправить в удалённый репозиторий
Создать Pull Request